

HANNS: Low-Storage Non-Interactive Approximate Private Nearest Neighbor Search with Sublinear Comparison Complexity

Haowen Pan*, Ruiqi Gan*, Yunhao Fu*, Yintai Sun*, Zhou Zhang*, Yuxiang Wang*,
Yi Chen[†], Bo Zhang[†], Haoyi Zhou*, Yongxin Tong*, Zhenyu Guan*, Jin Dong[†], and Song Bian*

**Beihang University, Beijing, China*

[†]Beijing Academy of Blockchain and Edge Computing, Beijing, China

Abstract—With growing concerns over data privacy, private nearest neighbors search (PNNS) attracts increasing research attention. Existing PNNS follow two main approaches: i) interactive PNNS based on secure multi-party computation protocols that leverage index structures to achieve sublinear complexity, and ii) non-interactive PNNS utilizing fully homomorphic encryption to minimize communication bandwidth at the cost of superlinear computational complexity. To address the communication-computation dilemma, we propose HANNS, a non-interactive PNNS protocol with a sublinear number of encrypted comparisons. Our key observation is that, while the full-table scan is inevitable under the non-interactive setting, the number of costly encrypted comparisons can be significantly reduced. Specifically, we develop a cluster ordering scheme over FHE that leverages a segmented rigid transformation to obliviously identify candidate clusters with only a sublinear number of homomorphic comparisons. Furthermore, we introduce a homomorphic product quantization (PQ) scheme that enables coarse search and reranking over PQ-encoded vectors, which significantly reduces the computational and storage overheads. In the experiment, we show that HANNS achieves $41\times$ to $277\times$ speedup and a storage reduction of $12\times$ to $31.7\times$ compared to the most recent non-interactive schemes, while reducing communication by $1,258\times$ to $80,536\times$ and achieving a speedup of $8\times$ to $119\times$ over interactive schemes in low-bandwidth scenarios.

1. Introduction

Nearest neighbors search (NNS) are commonly used to retrieve the closest matches to a query vector from a given vector set. NNS is the backbone of a plethora of data analytic applications such as recommendation systems [1], image recognition systems [2], retrieval-augmented generation (RAG) systems [3, 4], and many more [5–9]. In such applications, it is often the case that the queried data are stored remotely on cloud servers, where the data owner and the storage service provider are of different interest parties [10]. Consequently, the demands of low-cost and flexible data management [11–16] are met at the cost of data security, where sensitive vector data (e.g., private images, financial data) face the severe risks of data breaching [17, 18].

Facing the security challenges against data outsourcing, we see a line of recent literature that explores the use of cryptographic tools in constructing private nearest neighbors search (PNNS) schemes [4, 19–28]. Existing PNNS approaches can be broadly categorized into two types: i) interactive PNNS and ii) non-interactive PNNS. First, interactive PNNS typically relies on pre-built indices and employs a variety of secure multi-party computing (MPC) protocols to construct approximate nearest neighbor search (ANNS) schemes. By adopting an ANNS approach, interactive PNNS retrieves the approximate top- k closest neighbors to a given query input, rather than the exact ones. As a result, interactive PNNS schemes can be extremely fast for large-size encrypted vector search (EVS) [23, 24, 27], where the computational complexity is sublinear to the number of data items. Despite the efficiency in computation, interactive PNNS often incurs substantial communication overhead [4, 23, 27, 28], as the client needs to frequently communicate with the server to correctly traverse the pre-built index table of data clusters. In contrast, non-interactive PNNS [20–22, 25, 29, 30] are developed, primarily over fully homomorphic encryption (FHE) to reduce both the client-server interactions and communication bandwidth. However, to ensure data confidentiality, most (if not all) existing non-interactive PNNS schemes have to linearly scan the entire vector sets to compute and sort the similarities between the stored data vectors and the input vector, which constitutes an exact NNS scheme. However, since the best-known complexity for sorting over FHE is superlinear to the number of items, the induced computational overheads are prohibitive. For example, one of the most recent non-interactive PNNS [21] requires nearly three days to complete a Top-2 query over one million encrypted vectors.

Motivation: Given the computational overheads of exact PNNS, a natural question to ask is if we can also build ANNS over non-interactive cryptographic protocols with efficient querying performance. Here, we identify the key challenge in designing non-interactive ANNS. For cluster-based ANNS, the set of data vectors are first partitioned into multiple vector clusters. Then, the algorithm proceeds by calculating the Top- k matches only within the nearest clusters to the query vector, thereby avoiding a full-table scan. However, under a non-interactive setting, selectively accessing specific clusters inevitably leaks information about the distance range

of the query vector. To prevent information leakage, non-interactive PNNS typically needs to sort over all data vectors even under a clustered setting, resulting in the super-linear comparison complexity as mentioned above. To mitigate the performance penalty, our main observation is that, different FHE operators have drastically varied performance. Leveraging the performance difference, the goal here is to develop a cluster-based non-interactive ANNS that achieve sublinear complexity in terms of the computationally-heavy operators, but remain secure by scanning the entire vector dataset using lightweight operators.

1.1. Our Contribution

In this work, we propose HANNS, a non-interactive FHE-based ANNS framework with a sublinear number of comparisons. Specifically, we propose a three-stage homomorphic cluster-ordering scheme consisting of the cluster-center comparison stage, the inter-cluster permutation stage, and the intra-cluster comparison stage. Our core observation is that, by adopting the above three-stage architecture, we only need to perform the computationally intensive homomorphic comparison operation between the input query to a small amount of cluster centers (generally on the order of $\sqrt{n}$) and data vectors within candidate clusters (on the order of $k\sqrt{n}$), effectively reducing the comparison complexity from superlinear to sublinear. On the other hand, while the inevitable full-table scan does occur during the inter-cluster permutation stage, homomorphic vector permutation is mainly composed of lightweight and easily parallelizable homomorphic multiplications. Thus, by reducing the number of homomorphic comparisons, we can significantly speed up the PNNS process in practice. The main contributions of this work are summarized as follows.

- **A New FHE Framework for PNNS:** To the best of our knowledge, HANNS is the first non-interactive PNNS framework over FHE with sublinear comparison complexity. By leveraging the proposed three-stage homomorphic cluster-ordering scheme equipped with a homomorphic segmentation mechanism, we are able to support efficient encrypted vector search over large-scale vector databases.
- **Homomorphic Cluster Permutation:** While the cluster-ordering scheme can effectively reduce the number of homomorphic comparisons, the costs of homomorphically permuting vector clusters remain high. To mitigate such overheads, we propose a homomorphic rigid transformation algorithm to efficiently utilize the single-instruction-multi-data (SIMD) property of FHE to batch-swap the vector clusters.
- **Homomorphic Vector Compression:** We develop a product quantization (PQ) encoding and decoding mechanism tailored for encrypted vectors to reduce the storage and query overheads. To enable efficient PQ decoding under encryption, we introduce a plaintext-ciphertext comparison algorithm based

on Chebyshev bivariate approximation, which significantly improves decoding efficiency.

- **Thorough Experiments:** We show that, compared to the state-of-the-art (SOTA) non-interactive PNNS schemes, HANNS achieves a speedup of $41\times$ – $277\times$, and reduces the storage overhead by $12\times$ – $31.7\times$. Compared to SOTA interactive PNNS schemes, HANNS achieves a reduction of $1,258\times$ – $80,536\times$ in communication bandwidth, and is $8\times$ – $119\times$ faster in low-bandwidth scenarios.

1.2. Related Works

PNNS lies at the core of privacy-preserving vector similarity search. In typical scenarios, a client queries a remote database to retrieve vectors most similar to its encrypted input, while ensuring that neither the query nor the database content is exposed. This challenge has driven the development of numerous protocols and systems [4, 19–25, 27, 29, 31–35], with a wide variety of cryptographic techniques employed to balance privacy, efficiency, and scalability. In Table 1, we summarize the main characteristics of existing PNNS frameworks. Broadly, these approaches can be divided into two categories based on their interaction model: interactive PNNS and non-interactive PNNS.

1.2.1. Interactive PNNS. Interactive PNNS [4, 19, 23, 27, 29, 31–33, 35] is primarily designed to support ANNS, which offers a practical alternative to exact k -NNS in large, high-dimensional databases where exact search is often computationally prohibitive. These schemes are typically built under the two-party security model, where both the database server and the query client are unwilling to reveal their sensitive data to each other. To enable privacy-preserving ANNS, interactive PNNS leverage pre-built indexes to quickly identify candidate clusters through coarse search and perform linear scans only within these clusters, achieving sublinear computational overhead. A central mechanism of interactive PNNS is the subset retrieval, accessing only a small portion of the dataset to avoid scanning all encrypted records. To preserve privacy during this selective access, techniques such as locality-sensitive hashing (LSH) [24, 35], garbled circuits (GC) [36], secret sharing (SS) [37] oblivious RAM (ORAM) [38] and private information retrieval (PIR) [39] are employed. However, these cryptographic tools introduce significant communication overhead, which often becomes the main performance bottleneck. Furthermore, distributing secrets among participating parties in MPC-based techniques also leads to substantial requirements in communication bandwidth and interaction complexity [40–43].

1.2.2. Non-Interactive PNNS. Non-interactive PNNS has gained increasing attention for its ability to eliminate client-server interaction, significantly reducing communication overhead. These schemes are often deployed in the outsourced computation model, where an untrusted cloud server performs computations on encrypted data outsourced by the client. However, unlike interactive schemes that support

TABLE 1: Qualitative comparison of existing PNNS systems.

	Engorgio [21]	Zuber [20]	Cong <i>et al.</i> [25]	Hong [22]	Grass [26]	Servan-Schreiber <i>et al.</i> [24]	SANNS [23]	PANTHER [27]	Ours
Non-Interactive	●	●	●	●	●	●	○	○	●
Security Model	Outsource	Outsource	Outsource	Outsource	Two-Party	Two-Party	Two-Party	Two-Party	Both*
Server Setting	Single	Single	Single	Single	Single	Double	Single	Single	Single
Communication	Low	Low	Low	Low	Low	Low	High	Medium	Low
ANNS Support	○	○	○	○	●	●	●	●	●
High-Precision Distance	○	○	○	○	○	○	○	●	●
Distance Complexity	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	†	$\mathcal{O}(\sqrt{n})$	$\mathcal{O}(\sqrt{n})$	$\mathcal{O}(\sqrt{n})$
Number of Comparison ⁺	$\mathcal{O}(n \log^2 k)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n \log^2 k)$	$\mathcal{O}(n \log^2 k)$	$\mathcal{O}(n)$	-	-	-	$\mathcal{O}(k \sqrt{n} \log^2 k)$

*HANNS supports both the outsourced model and the two-party secure computation model. [†]Since the distance computations are performed via LSH, we do not consider their complexity here. ⁺We only count the number of homomorphic comparisons.

sublinear ANNS via pre-built index, non-interactive PNNS schemes perform exact k -NNS through full-table scan, which uniformly accesses all encrypted data to ensure access pattern privacy.

Recent FHE-based frameworks [20, 21, 25, 44] adopt this approach by linearly scanning the entire encrypted database. While fully non-interactive, these schemes require a superlinear number of homomorphic comparisons to preserve privacy, resulting in substantial computational overhead. In addition to high computational cost, existing FHE-based frameworks also lack the capability to effectively bridge high-dimensional distance computation with high-precision homomorphic sorting. Although some recent FHE-based works [21, 22, 44] have significantly optimized the overhead of homomorphic sorting, they are not directly applicable to PNNS. The key limitation lies in their approach to achieving high precision. For instance, Engorgio [21] performs arbitrary-precision sorting by segmenting and quantizing values before encryption, then performing segment-wise comparisons and aggregating the results. However, in PNNS, distances are computed over high-dimensional vectors, and once segmented, the resulting values require inter-segment carry propagation. For example, a 128-dimensional vector with 8-bit entries can yield a 23-bit squared distance after accumulation. This carry effect breaks the independence assumption of segmented comparisons, making it infeasible to perform accurate comparisons on encrypted segmented distances. While Kim *et al.* [26] propose an FHE-based PNNS scheme over graph-based index, it requires iterative execution of costly homomorphic comparisons and pre-encoding of graph information. Moreover, it also inherits the core challenge of encrypted high-dimensional distance computation, where carry propagation disrupts comparison accuracy and scalability.

Another line of work [24, 34, 45, 46] explores non-interactive PNNS via a two-server model, where two non-colluding servers hold identical copies of the dataset. However, this assumption is often impractical in real-world scenarios, limiting the applicability of such solutions.

2. Background and Preliminaries

Throughout this paper, we use bold lowercase letters (e.g., $\mathbf{a}$) for vectors, tilde lowercase letters (e.g., $\tilde{a}$) for

polynomials, and bold uppercase letters (e.g., $\mathbf{A}$) for matrices. We use $\mathbf{a} \ll k$ to denote a k -step cyclic left rotation of the elements in a vector $\mathbf{a}$. We use $[\cdot]_p$ to denote modular reduction on p . We use λ to denote the security parameter, P the plaintext modulus, Q the ciphertext modulus, and N the lattice dimension. Let $\mathbb{Z}_Q$ be the set of integers modulo Q , we define R , R_Q , $R_{\mathbb{Q}}$, and $R_{\mathbb{R}}$ to denote $\mathbb{Z}(X)/(X^N+1)$, $\mathbb{Z}(X)/(X^N+1) \bmod Q$, $\mathbb{Q}(X)/(X^N+1)$, and $\mathbb{R}(X)/(X^N+1)$, respectively, where $S(X)$ is the univariate polynomial in X whose coefficients are from the set S . For a comprehensive list of notations and terminologies, please refer to Appendix A.

2.1. Discrete CKKS

HANNS employs the discrete (also known as the quantized) CKKS scheme, an extension of the standard RNS-CKKS framework [47, 54, 55]. To represent high-precision data using constant-size ciphertexts in the discrete CKKS scheme, the input plaintext is generally encoded into a set of low-precision data segments [21, 61, 64]. More concretely, given a real-valued plaintext polynomial $\tilde{m}(X) = \sum_{i=0}^{N-1} m_i X^i$, the encoding process first quantizes $\tilde{m}(X)$ into a high-precision integer polynomial $\tilde{\mathbf{m}}(X) = \sum_{i=0}^{N-1} \mathbf{m}_i X^i \in R_P$. To enable homomorphic operations, $\tilde{\mathbf{m}}(X)$ is decomposed into a set of $w = \lceil \log P / \log B_g \rceil$ low-precision segment polynomials, denoted as $\{\tilde{\mathbf{m}}_0, \dots, \tilde{\mathbf{m}}_{w-1}\}$. The j -th segment polynomial is defined as:

$$\tilde{\mathbf{m}}_j(X) = \sum_{i=0}^{N-1} \mathbf{m}_{i,j} X^i \in R_{B_g},$$

where the coefficients $\mathbf{m}_{i,j}$ are derived from the base- B_g decomposition of the high-precision integer coefficients $\mathbf{m}_i = \sum_{j=0}^{w-1} \mathbf{m}_{i,j} B_g^j$.

Subsequently, each segment polynomial $\tilde{\mathbf{m}}_j$ is encrypted independently using the underlying RNS-CKKS protocol. Given a secret key $\tilde{s} \in R_Q$, the ciphertext for the j -th segment is generated as:

$$\text{ct}_j = \text{RLWE}_{\tilde{s}}^{N,Q}(\tilde{\mathbf{m}}_j) = (-\tilde{a} \cdot \tilde{s} + \tilde{\mathbf{m}}_j + \tilde{e}, \tilde{a}),$$

where $\tilde{a} \in R_Q$ is sampled uniformly at random, and $\tilde{e}$ is drawn from a discrete error distribution χ_{noise} . Collectively, these components form the QRLWE ciphertext:

$$\widehat{\text{QRLWE}}(\tilde{m}) = (\text{ct}_0, \dots, \text{ct}_{w-1}).$$

The decryption process proceeds in reverse: the component ciphertexts are decrypted, and the resulting segments are merged and decoded to recover the result message $\tilde{m}(X)$.

2.2. Homomorphic Operators

2.2.1. Basic Homomorphic Operators. We primarily use the following basic homomorphic operators to evaluate arithmetic functions over RLWE ciphertexts.

- $+$, $-$ and $\cdot$: Homomorphic addition, subtraction and multiplication. We represent an RLWE ciphertext as a tuple of polynomials. Homomorphic addition and multiplication between ciphertexts are performed in a component-wise manner. Given two ciphertexts $\text{RLWE}_0 = (\tilde{b}_0, \tilde{a}_0)$ and $\text{RLWE}_1 = (\tilde{b}_1, \tilde{a}_1)$ encrypting plaintext messages $\tilde{m}_0$ and $\tilde{m}_1$, their addition is defined as $\text{RLWE}(\tilde{m}_0 + \tilde{m}_1) = \text{RLWE}(\tilde{m}_0) + \text{RLWE}(\tilde{m}_1) = (\tilde{b}_0 + \tilde{b}_1, \tilde{a}_0 + \tilde{a}_1)$. Homomorphic subtraction is defined similarly. For multiplication, given two ciphertexts $\text{RLWE}(\tilde{m}_0)$ and $\text{RLWE}(\tilde{m}_1)$, the product $\text{RLWE}(\tilde{m}_0) \cdot \text{RLWE}(\tilde{m}_1)$ yields a ciphertext that encrypts the product of the plaintexts, i.e., $\tilde{m}_0 \cdot \tilde{m}_1$.

- $\text{Rot}(\text{RLWE}(\tilde{m}), \mathbf{RK}, \text{step})$: The homomorphic rotation. Given a ciphertext $\text{ct} = \text{RLWE}(\tilde{m})$, the rotation key $\mathbf{RK}$, and an integer $k \in \mathbb{N}$, Rot outputs a ciphertext $\text{RLWE}(\tilde{m} \ll \text{step})$. We refer the reader to [67] for details.

- $\text{Poly}_\rho(\text{RLWE}(\tilde{m}))$: Polynomial evaluation over ciphertext. Given a ciphertext $\text{RLWE}(\tilde{m})$, the result of applying Poly_ρ is a ciphertext $\text{RLWE}(\rho(\tilde{m}))$ that encrypts $\rho(\tilde{m})$. For some polynomial ρ , $\text{Poly}_\rho(\text{RLWE}(\tilde{m}))$ denotes the homomorphic evaluation of ρ on the input ciphertext $\text{RLWE}(\tilde{m})$.

2.2.2. Homomorphic Top- k . Homomorphic Top- k HomTopK is a crucial component of PNNS. We adopt the homomorphic Top- k operator proposed in [21] along with its HomOrdApp operator to perform Top- k sorting and permutation. The operator is defined as:

$$\text{ct}_{res}, \mathbf{CM} \leftarrow \text{HomTopK}(\text{ct}, k),$$

which takes a ciphertext ct and an integer k as inputs. HomTopK returns two outputs: a ciphertext ct_{res} , where the top- k elements represent the Top- k results, and a permutation matrix $\mathbf{CM}$ that records all the permutation vectors generated during the Top- k process. We provide the detailed permutation process in Appendix C.

2.3. Plaintext ANNS

Extensive research has been developed to enhance vector search, including graph-based [69, 70], tree-based [71], hashing-based [72, 73], and clustering-based [74] techniques. Among them, graph-based methods such as HNSW [69] typically offer the best performance in plaintext settings.

However, these graph-based methods are poorly suited for secure computation. Their data-dependent memory access patterns necessitate expensive linear searches to ensure data privacy, negating the efficiency of graph structures. In contrast, although clustering-based methods may perform moderately in plaintext scenarios, they are far more compatible with secure computation frameworks, which has also been observed in [23, 27]. Motivated by this, we adopt clustering-based ANNS, specifically the inverted file with product quantization (IVFPQ) [74] approach as the underlying plaintext search strategy in HANNS. In this work, we use the squared Euclidean distance as the distance metric, which is widely adopted in PNNS systems [21, 23, 25–27]. A typical IVFPQ-based ANNS workflow is shown in Algorithm 1. The database $\mathcal{D}$ is partitioned into t clusters $\{\mathcal{C}_0, \dots, \mathcal{C}_{t-1}\}$, each containing up to l vectors. For each cluster $\mathcal{C}_i$, we define its centroid as $\mathbf{c}_i \in \mathcal{C}$, along with a list of vector IDs $\{\text{ID}_{\mathcal{C}_0}, \dots, \text{ID}_{\mathcal{C}_{t-1}}\}$, and the PQ codes of residual vectors, where each residual vector $\mathbf{r}_{i,j}$ is computed as $\mathbf{v}_{i,j} - \mathbf{c}_i$ for vector $\mathbf{v}_{i,j} \in \mathcal{C}_i$, $j \in [0, l-1]$. In practice, the number of clusters t is typically set to approximately $\sqrt{n}$, resulting in a two-level structure where both the number of clusters and the number of vectors per cluster are on the order of $\sqrt{n}$. During preprocessing, each residual vector $\mathbf{r}_{i,j} \in \mathbb{R}^d$ is divided into l equal-length subvectors: $\mathbf{r}_{i,j} = [\mathbf{r}_{i,j}^{(0)}, \dots, \mathbf{r}_{i,j}^{(l-1)}]$, and each subvector $\mathbf{r}_{i,j}^{(z)}$ is quantized using a residual subspace trained by k -means [75, 76]:

$$\mathcal{CB}^{(z)} = \{\mathbf{b}_0^{(z)}, \dots, \mathbf{b}_{t-1}^{(z)}\}$$

where $\mathbf{b}_i^{(z)}$ is a d/l length vector. The PQ-encoded vector $\mathbf{pv}_{i,j} \in \mathbf{PV}$ of $\mathbf{r}_{i,j}$ is represented as $\mathbf{pv}_{i,j} = PQ(\mathbf{r}_{i,j}) = [id_0, \dots, id_{l-1}]$, where $id_z = \arg \min_i \|\mathbf{r}_{i,j}^{(z)} - \mathbf{b}_i^{(z)}\|^2$. At query time, given a query vector $\mathbf{q}\mathbf{v}$, we first compute the squared Euclidean distance between $\mathbf{q}\mathbf{v}$ and each cluster centroid $\mathbf{c}_i$ as $\|\mathbf{q}\mathbf{v} - \mathbf{c}_i\|^2$ and select the top- u nearest clusters. Given each selected cluster centroid $\mathbf{c}_i$, the residual vector relative to the query $\mathbf{q}\mathbf{v}$ is computed as $\mathbf{q}^{(i)} = \mathbf{q}\mathbf{v} - \mathbf{c}_i$. We then split each $\mathbf{q}^{(i)}$ into subvectors and build distance tables:

$$\mathcal{DT}_i^{(z)}[r] = \|\mathbf{q}^{(i,z)} - \mathbf{b}_r^{(z)}\|^2,$$

where $i \in [0, u-1]$, $z \in [0, l-1]$, and $r \in [0, t-1]$. For every PQ-encoded residual vector in cluster $\mathcal{C}_i$ with code $[id_0, \dots, id_{l-1}]$, we retrieve the corresponding distance values from the subspace distance tables. All candidates are then ranked by their distances, and the top- k IDs are returned.

3. Framework Overview

3.1. Motivation and Observation

We identify two key challenges that motivate the design of HANNS, namely, accuracy and efficiency. In what follows, we explain the concrete technical challenges and observations.

Algorithm 1: Plaintext IVFPQ-based ANNS

Input : Query vector $\mathbf{q}\mathbf{v} \in \mathbb{R}^d$, cluster centroids $\mathcal{C} = \{\mathbf{c}_0, \dots, \mathbf{c}_{t-1}\}$, residual PQ codebooks $\mathcal{CB} = \{\mathcal{CB}^{(0)}, \dots, \mathcal{CB}^{(t-1)}\}$, PQ-encoded database $\mathbf{P}\mathbf{V}$ contains t clusters $\mathcal{C}_i$, each contains (PQ code, ID) pairs. Search parameter u and k .

Output : Top- k approximate nearest neighbor IDs.

▷ Top- u Coarse search

```
1 for  $i = 0$  to  $t - 1$  do
2    $d_i \leftarrow \|\mathbf{q}\mathbf{v} - \mathbf{c}_i\|^2$ 
3 end
4  $\{\mathcal{C}_0^{\text{top}}, \dots, \mathcal{C}_{u-1}^{\text{top}}\} \leftarrow \text{Top-}u(\{d_i\}, \{\text{cluster}_i\})$ 
5   ▷ PQ decoding
6  $D_{\text{all}} \leftarrow [], I_{\text{all}} \leftarrow []$ 
7 for  $i = 0$  to  $u - 1$  do
8    $\mathcal{C} \leftarrow \mathcal{C}_i^{\text{top}}$ 
9    $\mathbf{c} \leftarrow \text{centroid of } \mathcal{C}$ 
10   $\mathbf{q} \leftarrow \mathbf{q}\mathbf{v} - \mathbf{c}$ 
11  Split  $\mathbf{q} = [\mathbf{q}^{(0)}, \dots, \mathbf{q}^{(t-1)}]$ 
12  for  $z = 0$  to  $t - 1$  do
13    for  $r = 0$  to  $\ell - 1$  do
14       $\mathcal{DT}^{(z)}[r] \leftarrow \|\mathbf{q}^{(z)} - \mathbf{b}_r^{(z)}\|^2$ 
15    end
16  end
17  for each  $([i_0, \dots, i_{t-1}], \text{ID})$  in  $\mathcal{C}$  do
18     $d \leftarrow \sum_{z=0}^{t-1} \mathcal{DT}^{(z)}[i_z]$ 
19    Append  $d$  to  $D_{\text{all}}$ , append ID to  $I_{\text{all}}$ 
20  end
21 end
22 ▷ Top- $k$  Reranking
23  $\{ID_0, \dots, ID_{k-1}\} \leftarrow \text{Top-}k(D_{\text{all}}, I_{\text{all}})$ 
Return :  $\{ID_0, \dots, ID_{k-1}\}$ 
```

Challenge and Observation 1: Non-interactive PNNS must process the entire dataset to preserve access pattern privacy. This typically involves a superlinear number of costly homomorphic comparisons during uniform searching and sorting over all encrypted vectors, resulting in substantial computational overhead. We observe that, while full data access is inevitable in non-interactive setting, the uniform application of costly homomorphic comparisons is not. In particular, homomorphic comparisons used for global permutation can be replaced with lightweight and parallelizable homomorphic multiplications. Building on this insight, HANNS transforms full-database operations into more efficient forms by using homomorphic multiplication to perform permutation, and applying PQ to compress vector dimensionality, which significantly reduces the computation and storage overhead of PNNS.

Challenge and Observation 2: Homomorphic cluster ordering needs to perform homomorphic sorting over high-precision distances, which are incompatible with the low-precision homomorphic comparison operators involved in the sorting process. To improve the sorting precision, existing

schemes [21, 77] encode high-precision plaintexts into multiple low-precision segments and perform homomorphic sorting over these segmented ciphertexts. However, current homomorphic distance computation methods [21, 25] cannot handle the carry propagation introduced during segmented arithmetic operations, resulting in limited precision or incorrect outputs. We observe that enabling non-interactive PNNS requires a segmented computation mechanism that correctly handles cross-segment carry propagation, effectively bridging homomorphic distance computation and cluster ordering.

3.2. Threat Model and Security

HANNS supports both outsourced and two-party computation security models. In the outsourced computation setting, our goal is to protect the outsourced database $\mathcal{D}$ owned by the client $\mathcal{C}$ from the semi-honest server $\mathcal{S}$. The concrete public and private data inside the server's scope is summarized as follows.

Public Data:

- n : The size of the database, i.e., the number of vectors in $\mathcal{D}$.
- d, w, B_g : The degree of the vector (e.g., picture = 1024 for a 32×32 -pixel picture vector), the number of quantized segments for the vector, and the precision parameter.
- l, t : The size of each cluster and the number of clusters.
- $\ell, \mathfrak{k}$: The number of PQ subspaces, and codebook size per subspace.
- u, k : The number of clusters selected during the coarse search and the number of results finally returned.

Private Data:

- $v_{i,j}$, for $i \in n, j \in d$: Exact values of the vector items in all $\mathbf{v} \in \mathcal{D}$.
- $qv_i \in \mathbf{q}\mathbf{v}$, for $i \in d$: Exact values in the query vector.

HANNS also supports the two-party security model. In this setting, only minor modifications are required: the preprocessing phase is shifted to the server side, and only the query vector needs to be encrypted. All subsequent operations remain fully consistent with those in the outsourced computation model. This design ensures mutual privacy: the client learns nothing about the database beyond the query result, while the server gains no information about the query vector. In the following sections, we present HANNS under the outsourced model setting.

Security of HANNS: Similar to that in previous works [26, 77–83], the private data owned by the client are all encrypted using FHE. Hence, the security of HANNS can be directly reduced to that of the underlying FHE scheme. In particular, the CKKS scheme guarantees security under chosen plaintext attacks, which translates to the security of HANNS against a semi-honest server.

3.3. HANNS Workflow

HANNS involves four main phases to process a PNNS query: database preprocessing, client data encryption, client query encryption, and server query evaluation. Below, we

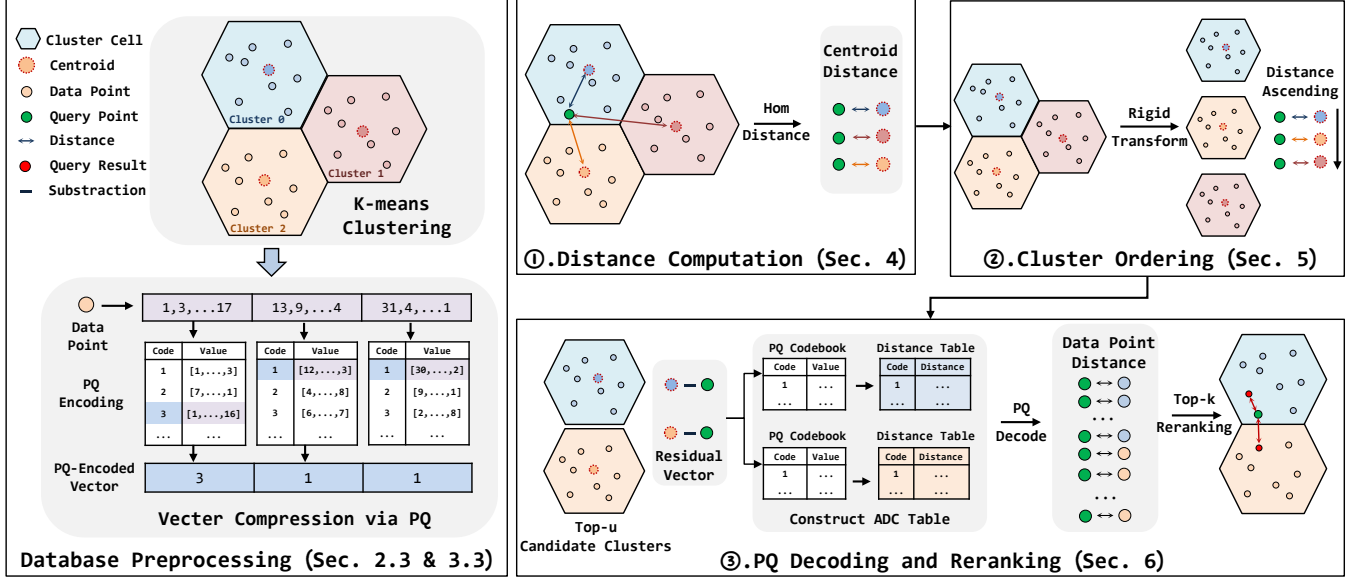


Figure 1: An example of processing an ANNS query over HANNS. Here, we demonstrate how to run a Top-2 query that sets $u = 2$ and combines PQ compression.

provide a concise overview of the main operations involved in each phase, which are also depicted in Figure 1.

• **Database preprocessing:** For a vector database $\mathcal{D}$ containing n vectors of dimension d , the client first constructs IVFPQ index locally on the database. The IVFPQ index is configured using three parameters: the number of coarse clusters t , the number of PQ subspaces l , and the codebook size per subspace ℓ . The index is constructed as described in Section 2.3, and all clusters are padded to the same size l . To reduce the storage expansion caused by padding, we follow the approximate balanced clustering method proposed in [23], which helps maintain cluster size uniformity while preserving overall partitioning quality. After the index is built, the client obtains $\mathcal{PD}$, which contains the following:

1. t clusters, each containing l PQ-encoded vectors of dimension l ;
2. t cluster centroid vectors, each of dimension d ;
3. A PQ codebook consisting of l sub-codebooks, each containing ℓ codewords, and each codeword is a vector of dimension d/l .

• **Client Data Encryption:** In the encryption phase, the client processes and encrypts the PQ-encoded vectors, cluster centroids, and PQ codebook in $\mathcal{PD}$. Each component is packed and expanded to fully utilize ciphertext slots and support batch queries. The encryption procedures for each component are summarized as follows:

i) PQ-encoded vectors $[\mathbf{PV}]$: assume the database is divided into t clusters, each containing h PQ-encoded vectors. For each fixed vector position $i \in [0, l-1]$, the client collects PQ-encoded vectors:

$$\{\mathbf{pv}_{0,i}, \mathbf{pv}_{1,i}, \dots, \mathbf{pv}_{t-1,i}\}$$

from all clusters. For each subspace $j \in [0, l-1]$, the client

extracts the j -th component of these vectors:

$$\{\mathbf{pv}_{0,i}[j], \mathbf{pv}_{1,i}[j], \dots, \mathbf{pv}_{t-1,i}[j]\}$$

and replicate this list as needed to fill the ciphertext slot space. The PQ-encoded vectors is then vertically packed and encrypted into RLWE ciphertexts.

ii) Cluster centroids $[\mathcal{C}]$: each centroid vector $\mathbf{c}_r \in \mathcal{C}$ of dimension d is first discretized into multiple low-precision integers using the encoding method proposed in [21] and segmented as $\mathbf{c}_r = \sum_{i=0}^{w-1} \mathbf{c}_{r, \text{seg}_i} B_g^i$. Then, for each dimension $j \in [0, d-1]$, the client collects the j -th elements of all segmented centroids $\mathbf{c}_{r, \text{seg}_i}[j]$, replicates and encrypts them into QRLWE ciphertexts.

iii) PQ codebook $[\mathcal{CB}]$: the codebook consists of l sub-codebooks, each containing ℓ code vectors of dimension d/l . $d/l \times \ell$. For each codeword ID $r \in [0, \ell-1]$, the client collects the r -th codeword $\mathbf{b}_r^{(z)}$ from each sub-codebook z and segments them as $\mathbf{b}_r^{(z)} = \sum_{i=0}^{w-1} \mathbf{b}_{r, \text{seg}_i}^{(z)} B_g^i$. The resulting set of vectors is repeated u times, where u is the number of clusters selected in coarse search (i.e., Top- u). The vectors are then encrypted into QRLWE ciphertexts.

Then the encrypted database $[\mathcal{PD}]$, containing encrypted PQ-encoded vectors $[\mathbf{PV}]$, cluster centroids $[\mathcal{C}]$, and PQ codebook $[\mathcal{CB}]$, along with the associated keys (i.e., rotation key $\mathbf{RK}$ and bootstrapping key $\mathbf{BK}$) are sent to the server.

• **Client Query Encryption:** For a query vector $\mathbf{qV}$, similar to the encryption of cluster centroids, the vector is segmented as $\mathbf{qV} = \sum_{i=0}^{w-1} \mathbf{qV}_{\text{seg}_i} B_g^i$, and subsequently encrypted into a QRLWE ciphertext denoted by $[\mathbf{qV}]$, which is then transmitted to the server. Note that the client can batch up to N/d query vectors within a single ciphertext.

• **Server Query Evaluation:** In the query evaluation phase, the server carries out PNNS on the encrypted database $[\mathcal{PD}]$ with the query vector $[\mathbf{qV}]$. The core steps involved in

the homomorphic evaluation of the PNNS are ① homomorphic distance computation, ② homomorphic cluster ordering, and ③ PQ decoding and reranking.

During the homomorphic distance computation, the encrypted query vector $[\mathbf{q}\mathbf{v}]$ and each cluster centroid are processed through segmented homomorphic subtraction, multiplication, and addition. These operations yield intermediate segmented results. Since the resulting intermediate values are segmented and lack proper carry propagation, they cannot be directly used for comparison or sorting. To address this, a homomorphic segment computation mechanism is employed, incorporating a homomorphic carry propagation operator HomCarry . This operator aggregates the partial results and reconstructs the final segmented distance ciphertext with resolved carries, enabling correct numerical ordering. Once the distances between the query vector $[\mathbf{q}\mathbf{v}]$ and the cluster centroids $[\mathbf{c}_i]$ are obtained, the server proceeds to sort the clusters. To enable efficient candidate selection without disrupting the cluster structure, a homomorphic rigid transformation operator HomRT is used to efficiently sort the cluster centroids while preserving their structural alignment with corresponding PQ-encoded vectors. Then, the server extracts the first u encrypted cluster centroids in $[\mathcal{C}]$ after cluster ordering to get the Top- u nearest centroids. Since the homomorphic rigid transformation maintains the structural alignment between cluster centroids and their respective encrypted elements in $[\mathbf{P}\mathbf{V}]$, the server can directly retrieve the Top- u candidate vectors by extracting the first u PQ-encoded vectors in $[\mathbf{P}\mathbf{V}]$ after transformation. After homomorphic cluster sorting, the server constructs distance lookup tables based on the residual vectors between the query vector and the selected candidate cluster centroids, which are then used for PQ decoding. Subsequently, homomorphic subspace distance retrieval is applied to decode each candidate PQ-encoded vector and compute its corresponding ADC distances. Based on the ADC distances, the server performs reranking and returns the Top- k closest results.

4. Homomorphic Distance Computation

In this section, we focus on the homomorphic high-dimensional vector distance computation. We begin by analyzing the inherent incompatibility between segmented encoding and distance computation. Then, we introduce a segmented computation mechanism designed to accurately manage the carry propagation during segmented computation.

4.1. High-Precision Distance Computation

Distance computation is the key operation in PNNS, as it directly reflects vector similarity. However, existing homomorphic distance computation methods face limitations in both precision and usability.

For example, TFHE-based approaches [25] support only low-precision computation (up to 9 bits) due to the inherent constraints of programmable bootstrapping [84]. Additive schemes like BFV [85] have also been used [23, 27]; however, due to the inefficiency of BFV bootstrapping, subsequent

comparison and sorting operations are typically offloaded to GC [36] or SS [37], which are not suitable for non-interactive settings. In CKKS-based schemes [21, 22], the precision of distance computation is tightly coupled with that of homomorphic comparisons. To improve homomorphic comparison precision while mitigating noise accumulation, high-precision values are often mapped to multiple low-precision segments. For example, given messages x and y , they are segment into (x_0, x_1) and (y_0, y_1) , where $x = \sum x_i B_g^i$ and $y = \sum y_i B_g^i$. To compare $x > y$, [21] make comparison on each part: $x_0 > y_0$, $x_1 > y_1$, and $x_1 == y_1$. The result is selected as $(x_1 == y_1) ? (x_0 > y_0) : (x_1 > y_1)$. This segmented strategy improves comparison efficiency and precision. However, such methods face challenges in PNNS where linear operations (e.g., addition or multiplication) often precede comparisons. Segmented encodings lack cross-segment carry propagation, which can lead to incorrect results. For instance, to compare x^2 and y^2 , segment-wise squaring produces x_0^2, x_1^2 and y_0^2, y_1^2 . Applying the same logic of comparing $x_0^2 > y_0^2$ and $x_1^2 > y_1^2$ while selecting based on $x_1^2 == y_1^2$ fails to reflect the actual result of $x^2 > y^2$, because the squared values may result in carries across segments that are not accounted for in the segmented comparison.

To bridge this gap between high-dimensional distance computation and homomorphic comparison, we propose a segmented computation mechanism equipped with a homomorphic carry propagation operator to support accurate segmented distance computation in high-dimensional vector spaces. Specifically, our distance computation begins with segmented homomorphic subtraction and squaring between the segmented query vector $\mathbf{q}\mathbf{v}_{\text{seg}_i}$ and the r -th segmented cluster centroid $\mathbf{c}_{r,\text{seg}_i}$. For each vector dimension $j \in [0, d-1]$, we compute the intermediate squared distance values across segments as: $\{\mathbf{td}_0, \dots, \mathbf{td}_{w-1}\}$, where $\mathbf{td}_i = (\mathbf{q}\mathbf{v}_{\text{seg}_i}[j] - \mathbf{c}_{r,\text{seg}_i}[j])^2$. We then apply homomorphic dynamic segment multiplication HomSegMul to each $\mathbf{td}_i$ to reconstruct the carried intermediate distances. These results are subsequently summed homomorphically across all dimensions, and a final homomorphic carry propagation is applied to obtain the full-precision distance $\mathbf{d}$.

4.2. Segmented Computation Mechanism

We observe that homomorphic segmented computation is the key to achieving high precision distance computation. Although [64] proposes a segmented computation mechanism for discrete CKKS, which performs computation over a finite field. However, distance computation in many applications is followed by a comparison step, and performing carry operations within a finite field may disrupt the numerical ordering between distances, leading to incorrect comparison results. To address this, we design a discrete segmented computation mechanism that supports dynamic precision adjustment during computation, enabling accurate segmented distance computation in high-dimensional vector spaces. We use segmented multiplication as a representative example to demonstrate our segmented computation mechanism with

Algorithm 2: Homomorphic Carry HomCarry

Input : Precision parameter B_g , and a QRLWE ciphertext ct that contains encrypted segments potentially exceeding B_g .

Output : Two QRLWE ciphertext ct_{low} and ct_{high} encrypts the residual part and carried part.

- 1 $ct_{\text{low}} \leftarrow \text{HomMod}(ct, B_g)$
- 2 $ct_{\text{sub}} \leftarrow ct - ct_{\text{low}}$
- 3 $ct_{\text{high}} \leftarrow ct_{\text{sub}} \cdot 1/B_g$

Return : $ct_{\text{low}}, ct_{\text{high}}$

adjustable precision, as detailed in Algorithm 3. The same mechanism applies to addition as well.

Specifically, given two segmented messages of the form $x = x_0 + x_1 B_g$ and $y = y_0 + y_1 B_g$, where $x_0, x_1, y_0, y_1 \in \mathbb{Z}_{B_g}$, the product xy expands to $xy = x_0 y_0 + (x_0 y_1 + x_1 y_0) B_g + x_1 y_1 B_g^2$. This result exceeds the modulus range of each segment and must be resegmented homomorphically. Line 1–10 in Algorithm 3 compute all pairwise products $x_i y_j$ and accumulate them into raw segments. Each resulting term $ct_{\text{raw}}[i]$ may exceed B_g , so we invoke HomCarry (Line 12–14) to resegment these ciphertexts. As shown in Algorithm 2, the HomCarry procedure takes an input ciphertext ct and returns two outputs in $\mathbb{Z}_{B_g}$:

$$\begin{aligned} (ct_{\text{low}}, ct_{\text{high}}) &= \text{HomCarry}(ct, B_g) \\ &= \left(\text{HomMod}(ct, B_g), \frac{ct - ct_{\text{low}}}{B_g} \right). \end{aligned}$$

Each raw segment is thus split into a low and high component. To reconstruct the final segmented multiplication result, we perform homomorphic carry propagation across segment levels. For each position i , we add the high component of segment $i-1$ to the low component of segment i and reapply HomCarry to this sum. The resulting low part becomes the i -th output segment, while the high part propagates to the next level. Line 15–21 implement this iterative carry propagation and final reassembly. The output is a $2w$ -segment ciphertext representing the product $x \cdot y$ with precision-preserving resegmentation. By dynamically increasing the number of segments (e.g., from w to $2w$), we can homomorphically support higher-precision multiplication results without changing the underlying logic.

5. Homomorphic Coarse Search

In this section, we present the coarse search in HANNS, which identifies the Top- u centroids based on the distance between the query vector and cluster centroids. The corresponding candidate clusters are then selected for reranking to produce the final Top- k results.

5.1. Homomorphic Cluster Ordering

After obtaining the distances between the query vector and the cluster centroids through method proposed in Section 4, we proceed to the cluster-center comparison stage and

Algorithm 3: Homomorphic Dynamic Segment Multiplication HomSegMul

Input : Precision parameter B_g , the number of segmentation w , and two QRLWE

$$ct_x = \{\text{RLWE}(\tilde{x}_0), \dots, \text{RLWE}(\tilde{x}_{w-1})\}$$

$$ct_y = \{\text{RLWE}(\tilde{y}_0), \dots, \text{RLWE}(\tilde{y}_{w-1})\}.$$

Output : A QRLWE ciphertext

$$ct_{\text{res}} = \{\text{RLWE}(\tilde{r}_0), \dots, \text{RLWE}(\tilde{r}_{2w-1})\}$$

encrypts the result $ct_x \cdot ct_y$ with carry propagation.

- 1 $ct_{\text{raw}}, ct_{\text{res}}, ct_l, ct_h \leftarrow \{\}$
- 2 **for** $i = 0$ **to** $w - 1$ **do**
- 3 **for** $j = 0$ **to** $w - 1$ **do**
- 4 $ct_{\text{prod}} \leftarrow ct_x[i] \cdot ct_y[j]$
- 5 **if** $i \neq j$ **then**
- 6 $ct_{\text{prod}} \leftarrow 2 \cdot ct_{\text{prod}}$
- 7 **end**
- 8 $ct_{\text{raw}}[i + j] + = ct_{\text{prod}}$
- 9 **end**
- 10 **end**
- 11 **for** $i = 0$ **to** $2w - 2$ **do**
- 12 $ct_{\text{low}}[i], ct_{\text{high}}[i] \leftarrow \text{HomCarry}(ct_{\text{raw}}[i])$
- 13 **end**
- 14 $ct_{\text{carry}} \leftarrow ct_h[0]$
- 15 $ct_{\text{res}}[0] \leftarrow ct_l[0]$
- 16 **for** $i = 1$ **to** $2w - 2$ **do**
- 17 $ct_{\text{tmp}} \leftarrow ct_{\text{carry}} + ct_l[i]$
- 18 $ct_{\text{res}}[i], ct_{\text{carry}} \leftarrow \text{HomCarry}(ct_{\text{tmp}}, B_g)$
- 19 $ct_{\text{carry}} \leftarrow ct_{\text{carry}} + ct_h[i]$
- 20 **end**
- 21 $ct_{\text{res}}[2w - 1] \leftarrow ct_{\text{carry}}$

Return : ct_{res}

the inter-cluster permutation stage, which sorts the Top- u nearest clusters and retrieve all PQ-encoded vectors within these u clusters. Many prior works [21, 22, 25, 30, 44, 77, 86, 87] have focused on optimizing the homomorphic Top- k (or sorting) algorithm, as an efficient Top- k algorithm can significantly accelerate the identification of the nearest clusters. However, in the context of non-interactive PNNS, we observe that the cost of retrieving the elements corresponding to the selected clusters far exceeds the cost of Top- k sorting itself. This is primarily due to the scale disparity between the number of cluster centroids (on the order of $\sqrt{n}$) and the full database (size n). To address this, we adopt the current SOTA homomorphic Top- k algorithm [21] to efficiently sort the cluster centroids, and instead focus our efforts on optimizing the retrieval of elements associated with the selected clusters. A straightforward approach is to maintain an explicit mapping from each database element v to its associated cluster centroids c . Then, during query execution, all elements can be homomorphically compared against the selected Top- u cluster centroids. Valid matches are marked with 1 and others with 0, allowing the system to filter out irrelevant elements. This operation is conceptually similar to the JOIN operation in database, which results in a superlinear

number of homomorphic comparisons across all database entries.

To address this problem, we propose a homomorphic cluster ordering scheme, which consists of two main steps: cluster-center comparison for sorting Top- u cluster centroids and inter-cluster permutation for globally permuting clusters based on the Top- u results. The detailed algorithm for homomorphic cluster ordering is provided in Algorithm 4. In the first step, we apply the `HomTopK` and `HomOrdApp` to perform Top- u sorting over the encrypted cluster centroids. This yields the Top- u closest centroids along with a corresponding permutation matrix $\mathbf{CM}$ that captures the permutation vectors during sorting (Line 1-8). In the second step, we use the permutation vectors to perform homomorphic rigid transformation `HomRT` on the encrypted cluster elements (Line 9-13). As described in Section 3.3, all clusters are preprocessed to have a uniform size l , and elements at the same position across clusters are vertically packed. Leveraging the permutation vectors from `HomTopK`, we homomorphically permute each ciphertext to align cluster elements according to the sorted order. During this permutation, we prune invalid clusters and repurpose the resulting vacant slots to merge valid elements from other clusters, thereby improving packing efficiency and reducing computational overhead.

5.2. Homomorphic Rigid Transformation

We design a homomorphic rigid transformation operator `HomRT` that leverages homomorphic multiplication to perform inter-cluster permutation.

As specified on Line 1–6 of Algorithm 5, in the first stage of `HomRT`, due to the presence of a large number of irrelevant clusters in the Top- u sorting process, we first replace the positions of these pruned clusters with elements from clusters that have potential to be candidates. This compaction halves the total number of ciphertexts to be processed. To align with the repacked cluster ciphertexts, we apply the same compaction strategy to the encrypted permutation matrix, which is derived from the comparison results at each stage of the bitonic sorting network [88], ensuring structural consistency as shown in Line 7-9. Since the permutation matrix encodes the comparison results of the bitonic sorting network, we adopt a matrix-based permutation technique to permute the cluster elements (Line 10–15), following the design in [21]. This approach combines homomorphic rotations and multiplications to efficiently rearrange ciphertext elements. Full details of the permutation process are provided in Appendix C. Above procedure is iteratively applied across transformation levels from 0 to $\log(l) - \log(u) - 1$, progressively narrowing down the candidate set. During this process, clusters that are not among the Top- u are pruned and excluded from further computation. To improve efficiency, the vacated ciphertext slots created by pruning are dynamically reused and repacked with valid elements, as illustrated in Figure 2, thereby reducing the overhead of ciphertext multiplication. After obtaining the

Algorithm 4: Cluster Ordering `HomOrder`

Input : A QRLWE ciphertext ct_d encrypts the distance between the query and the centroids. Encrypted PQ-encoded vectors $[\mathbf{PV}]$. Cluster size l and the number of candidate clusters u . Rotate key $\mathbf{RK}$.

Output : $[\mathbf{PV}_{cand}]$, which contains encrypted candidate PQ-encoded vectors.

```

1  $ct_{Topu}, \mathbf{CM} \leftarrow \text{HomTopK}(ct_d, u)$ 
2 for  $s = 0$  to  $\log(u)$  do
3   for  $p = 0$  to  $s$  do
4     for  $ct_{cv}$  in  $[\mathbf{PV}]$  do
5        $ct_{cv} \leftarrow \text{HomOrdApp}(ct_{cv}, \mathbf{CM}[s][p], s, p, \mathbf{RK})$ 
6     end
7   end
8 end
9  $[\mathbf{PV}_0] \leftarrow [\mathbf{PV}]$ 
10 for  $z = 1$  to  $\log(l) - \log(u) - 1$  do
11    $[\mathbf{PV}_z] \leftarrow \text{HomRT}([\mathbf{PV}_{z-1}], u, \mathbf{CM}[\log(u) + z + 1], \mathbf{RK})$ 
12 end
13  $[\mathbf{PV}_{cand}] \leftarrow [\mathbf{PV}_{\log(l) - \log(u) - 1}]$ 
Return :  $[\mathbf{PV}_{cand}]$ 

```

candidate cluster elements, we can perform intra-cluster comparison within the Top- u candidate clusters.

Through this method, we reduce the cost of locating candidate elements by replacing expensive homomorphic comparisons over the entire database with homomorphic comparisons limited to cluster centroids and candidate clusters, followed by a homomorphic rigid transformation applied across the entire database. Specifically, homomorphic Top- k requires $\mathcal{O}(n \log^2 k)$ comparisons in the naive KNNs setting, whereas the homomorphic rigid transformation only involves $\mathcal{O}(n \log^2 k)$ multiplications and rotations. In our approach, we avoid sorting the entire dataset. Instead, we only perform homomorphic Top- u over the $\sqrt{n}$ cluster centroids in the cluster-center comparison stage and perform homomorphic Top- k over the $u \cdot \sqrt{n}$ elements in the intra-cluster comparison stage. All other elements are handled through homomorphic rigid transformations in the inter-cluster permutation stage. As a result, the number of homomorphic comparisons is reduced from $\mathcal{O}(n \log^2 k)$ to $\mathcal{O}(\sqrt{n}(\log^2 u + u \log^2 k))$. Note that, since it is generally the case that u and k are same order of magnitude, while $u \ll \sqrt{n}$ and $k \ll \sqrt{n}$, thus we achieve a sublinear number of homomorphic comparisons (e.g., $\mathcal{O}(k\sqrt{n} \log^2 k)$).

6. Homomorphic Reranking

Following the coarse search, we proceed to the final stage of selecting the Top- k results from the Top- u candidate clusters. We first explain the procedure of PQ decoding in HANNS, and then describe the subspace distance retrieval.

Algorithm 5: Rigid Transformation HomRT

Input : The PQ encoding vectors $[\mathbf{PV}_z]$ after z -th transformation. Cluster size l and the number of candidate clusters u . The permutation matrix $\mathbf{CM}_z$ for z -th transformation. Rotate key $\mathbf{RK}$.

Output : $[\mathbf{PV}_{z+1}]$, which contains the PQ encoding vectors after $z + 1$ -th transformation.

```

1 for  $i = 0$  to  $l/2^{z+1}$  do
2   for  $j = 0$  to  $l$  do
3      $ct_{rot} \leftarrow \text{Rot}(\mathbf{PV}_z[i][j], \mathbf{RK}, 2u)$ 
4      $[\mathbf{PV}_{z+1}][i][j] \leftarrow ct_{rot} + \mathbf{PV}_z[l/2^{z+1} + i][j]$ 
5   end
6 end
7 for  $p = 0$  to  $\log(u)$  do
8    $\mathbf{CM}_{rot} \leftarrow \text{Rot}(\mathbf{CM}_z[p], \mathbf{RK}, 2u)$ 
9    $\mathbf{CM}_{z+1}[p] \leftarrow \mathbf{CM}_z[p] + \mathbf{CM}_{rot}$ 
10  for  $ct_{cv}$  in  $[\mathbf{PV}_{z+1}]$  do
11     $ct_{sup} \leftarrow \text{Rot}(ct_{cv}, \mathbf{RK}, -2^{\log(u)-p})$ 
12     $ct_{sub} \leftarrow \text{Rot}(ct_{cv}, \mathbf{RK}, 2^{\log(u)-p})$ 
13     $ct_{cv} \leftarrow ct_{sup} \cdot \mathbf{CM}_{z+1}[p][0] + ct_{cv} \cdot$ 
       $\mathbf{CM}_{z+1}[p][1] + ct_{sub} \cdot \mathbf{CM}_{z+1}[p][2]$ 
14  end
15 end
Return :  $[\mathbf{PV}_{z+1}]$ 

```

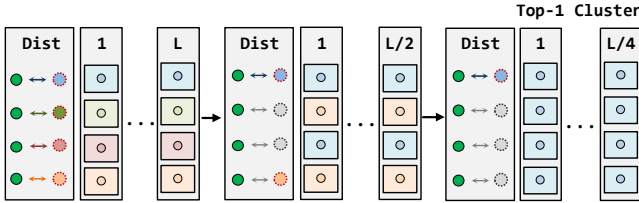


Figure 2: Example of homomorphic rigid transformation.

6.1. Homomorphic PQ decoding

After the coarse search, we obtain the PQ-encoded vectors $\mathbf{pv}_{i,j} \in \mathcal{C}_i^{\text{top}}$ in Top- u candidate clusters. However, the PQ codes $id_z \in \mathbf{pv}_{i,j}$ in PQ-encoded vectors cannot be directly used for reordering. PQ decoding is required to convert the PQ codes into ADC distances with respect to the query vector, which are then used for final reordering. As illustrated in Figure 3 and described in Section 2.3, the PQ decoding process consists of two main stages: distance table construction and subspace distance retrieval. In the distance table construction stage, we first compute the residual vectors by subtracting the centroids of the Top- u clusters from the query vector. Then, using the distance computation method described in Section 4, we build the distance tables for each subspace. In the subspace distance retrieval stage, we retrieve the distance corresponding to each code in the PQ-encoded vector from the appropriate row of the distance table. For each sub-code id_z in PQ-encoded vector $\mathbf{pv}_{i,j}$, we compare it with the indices in the corresponding subspace and select the associated distance value. Finally, all subspace distances

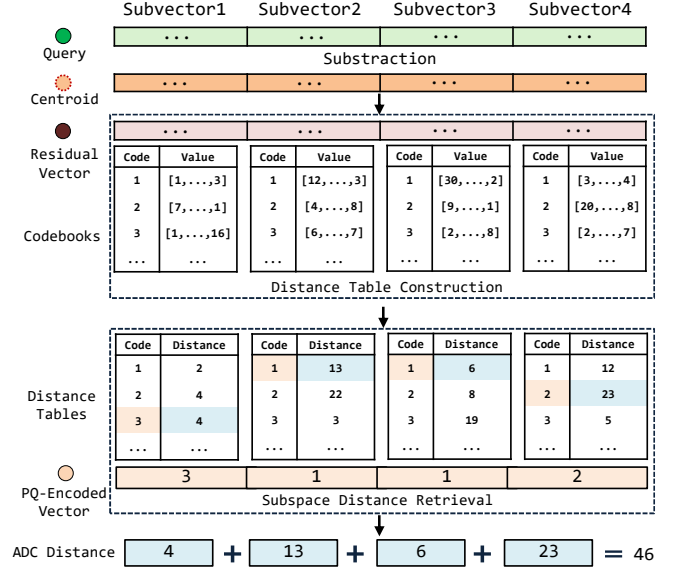


Figure 3: Demonstration of PQ decoding.

are summed to compute the ADC distance, which then used to perform the final reranking of candidate vectors.

Remark: PQ plays a central role in our design by offering two core optimizations: First, PQ reduces the dimension of each vector by partitioning the original d -dimensional vector into l subspaces. Each subspace is quantized into a compact code, converting 32-bit floats into a few bits, which significantly lowers the cost of homomorphic rigid transformation and ciphertext size. Second, PQ eliminates the need to perform homomorphic distance computation over all database vectors. Instead, distance evaluation is conducted only between the encrypted query and the compact PQ codebook. This enables efficient distance computation via lightweight lookup and accumulation, significantly reducing the overall computation cost.

6.2. Subspace Distance Retrieval

In the subspace distance retrieval stage of PQ decoding, each encrypted code $[id_0, \dots, id_l]$ in $\mathbf{pv}_{i,j}$ need to be compared with all possible plaintext indices $[0, \dots, \ell - 1]$ of the corresponding distance table to identify and extract the matching distances. The core operation in this process is homomorphic comparison. We observe that, unlike other comparison stages in the HANNS, this step involves comparisons between encrypted codes and plaintext indices. However, existing plaintext-ciphertext comparison schemes [81, 89, 90] typically require special encoding of the ciphertext, making them unsuitable for this case. To address this limitation and improve the efficiency of PQ decoding, we propose a novel plaintext-ciphertext comparison algorithm based on bivariate Chebyshev polynomial approximation [91].

Our core design insight stems from the observation that standard plaintext-ciphertext comparisons in CKKS [21, 92–95] are implemented using Chebyshev polynomial ap-

proximations [91]. Specifically, for a set of coefficients $C = \{c_0, c_1, \dots, c_n\}$, the approximated comparison function $f(x - y)$ is expressed as:

$$f(x - y) = \sum_i^n c_i T_i(x - y),$$

where $T_i(x)$ denotes the Chebyshev polynomial of the first kind. A typical comparison operation can thus be formulated as $\text{Poly}_f(ct_x - ct_y)$, where ct_x and ct_y are the two ciphertext to be compared. In our case, the scenario differs: we need to compare each encrypted PQ code value $[id_0, \dots, id_{t-1}] \in \mathbf{pv}_{i,j}$ against all possible plaintext indices $[0, \dots, \mathfrak{k} - 1]$. Since these indices are known in advance during preprocessing, we aim to decouple the variables x and y in the expression $f(x - y)$, where x is encrypted as ciphertext and y is the plaintext index. To achieve this, we approximate the comparison function $f(x - y)$ using a bivariate Chebyshev polynomial expansion:

$$f(x - a) = \sum_{i=0}^n \sum_{j=0}^n C_{i,j} T_i(y) T_j(x),$$

where $C_{i,j}$ denotes the matrix of approximation coefficients. This formulation allows all Chebyshev terms $T_i(y)$ in the plaintext domain to be precomputed offline. The resulting terms $C_{i,j} T_i(y)$ are thus fixed plaintext values. During the online phase, the ciphertext-dependent components $T_j(x)$ are evaluated homomorphically and shared across all comparisons for plaintext indices $[0, \dots, \mathfrak{k} - 1]$. The final evaluation is then performed by multiplying each $T_j(x)$ with the corresponding precomputed scalar $C_{i,j} T_i(y)$ via plaintext-ciphertext multiplication, which reduces the cost of online PQ-decoding in half. Moreover, since each plaintext-ciphertext product is independent, the entire comparison process supports highly parallel execution, enabling efficient batched evaluation of multiple plaintext-ciphertext comparisons. We provide the construction details of the bivariate Chebyshev approximation in Appendix B.

7. Evaluation

In evaluating HANNS, we first perform microbenchmark evaluations to demonstrate the performance improvements of each individual component in HANNS. Then we conduct end-to-end evaluations and compare HANNS with the SOTA non-interactive and interactive PNNS schemes.

TABLE 2: Clustering hyperparameters in HANNS.

Dataset	l	t	$\mathfrak{l}$	$\mathfrak{k}$	u
DEEP100K	512	256	16	256	16
SIFT1M	1024	2048	16	256	16
DEEP1M	2048	1024	16	256	16
DEEP10M	4096	4096	16	256	32

TABLE 3: Search quality comparison (Recall@1) on different datasets.

Dataset	$k = 16$	$k = 32$	$k = 64$	$k = 128$
DEEP100K	90.14%	91.81%	92.15%	92.19%
SIFT1M	90.08%	91.96%	92.64%	92.77%
DEEP1M	90.15%	93.28%	94.41%	94.74%
DEEP10M	87.27%	92.17%	94.40%	97.40%

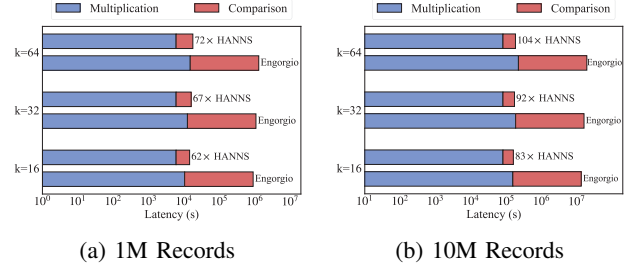


Figure 4: Breakdown of homomorphic Top- k search latency over 1M and 10M records.

7.1. Experiment Setup

•**Environment:** We implement HANNS based on OpenFHE [96] using C++17 and compiled with GCC 10.2.0. The experiments are carried out on an Intel Xeon Gold 6226R processor with 512 GB of RAM in a single-thread environment.

•**Datasets and Parameters:** We use four datasets of different sizes and dimensions for the experiment: DEEP100K ($n = 10^5$, $d = 96$), SIFT1M ($n = 10^6$, $d = 128$), DEEP1M ($n = 10^6$, $d = 96$) and DEEP10M ($n = 10^7$, $d = 96$). The clustering hyperparameters used in HANNS are shown in Table 2. While using a more efficient plaintext clustering algorithm or changing the hyperparameters might enhance the performance of HANNS, our main objective is to improve the fundamental performance of PNNS. We set the lattice dimension $D = 2^{17}$, the ciphertext slot size $N = D/2 = 2^{16}$, and the largest ciphertext modulus Q_L to be a 3,516-bit integer. The secret keys are drawn from a ternary distribution as specified in [97]. The parameters are consistent with [21] and guarantee 128-bit security according to [98, 99].

7.2. Microbenchmarks

In microbenchmark evaluation, we demonstrate the performance improvements of individual components in HANNS, including homomorphic comparison reduction, approximate homomorphic Top- k selection, and PQ-based vector compression. The results are summarized as follows.

•**Sublinear Homomorphic Comparison:** We compare the cost breakdown of homomorphic comparisons and multiplications in Top- k selection between HANNS and the most recent work Engorgio [21]. As shown in Figure 4, HANNS significantly reduces the proportion of homomorphic comparison cost across varying data sizes, achieving a 62× to 104× speedup over the baseline.

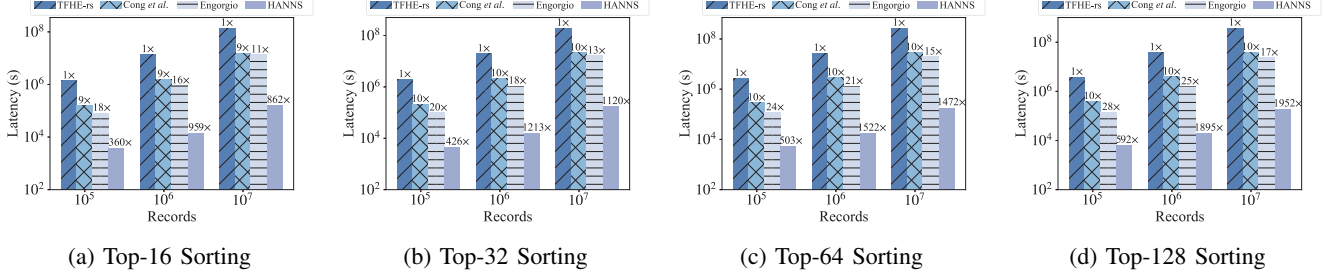


Figure 5: Comparison between our homomorphic approximate Top- k sorting and exact homomorphic Top- k sorting with varying sizes of records and different k values.

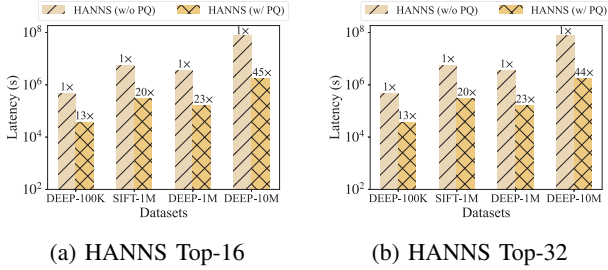


Figure 6: Comparison of HANNS with and without PQ compression across different datasets.

• **Homomorphic Approximate Top- k :** We compare the performance of our homomorphic approximate Top- k with existing exact homomorphic Top- k solutions [21, 25, 100] across various values of k and dataset sizes, where $n = 10^5$, 10^6 , and 10^7 correspond to the DEEP100K, DEEP1M, and DEEP10M datasets, respectively. As shown in Table 3, our method consistently achieves high recall rates across all configurations. Furthermore, Figure 5 shows that HANNS outperforms existing exact homomorphic Top- k solutions by a factor of $20\times$ to $1,951\times$.

• **PQ Compression:** We implemented two versions of HANNS: one with PQ compression and one without, denoted as HANNS (w/ PQ) and HANNS (w/o PQ), respectively. We then use both versions to perform PNNS across various k values and datasets. As shown in Figure 6, the PQ-enabled version achieves a $13\times$ to $44\times$ speedup compared to the version without PQ. Furthermore, Table 4 illustrates the storage optimization brought by our PQ compression, reducing the ciphertext storage size by $12\times$ to $31.7\times$ compared [21]. Note that both the computational and storage advantages become more pronounced as the dataset size and vector dimensionality increase, indicating that HANNS is well-suited for large-scale vector search scenarios.

7.3. End-to-End Evaluation

To evaluate the end-to-end performance of HANNS, we compare it with SOTA non-interactive and interactive PNNS schemes in terms of computational efficiency and communication cost.

TABLE 4: Storage analysis of 1M records with different precision and dimension.

Precision	d	Methods	Ciphertext Size	
16-bit	96	Engorgio [21]	36GB	$12\times$
		This Work	3GB	$1\times$
16-bit	128	Engorgio [21]	72GB	$23.8\times$
		This Work	3.01GB	$1\times$
32-bit	96	Engorgio [21]	48GB	$15.9\times$
		This Work	3.01GB	$1\times$
32-bit	128	Engorgio [21]	96GB	$31.7\times$
		This Work	3.02GB	$1\times$

We first compare the query latency of HANNS with existing non-interactive PNNS schemes [21, 25, 26] across different datasets and various k values. Since [26] is not publicly available, we use the Top-16 results from their paper. For larger values of k , we assume their performance remains approximately the same. As [25] supports only 6-bit distance PNNS, we estimate the cost of 24-bit support by scaling its overhead by $10\times$, following the result in [26, 83]. As shown in Figure 7, HANNS achieves $41\times$ to $277\times$ speedup over existing non-interactive solutions across different datasets.

As mentioned in Section 3.2, HANNS also supports the two-party security model. Therefore, we compare its performance with existing interactive PNNS schemes [23, 27]. As shown in Table 5, HANNS achieves a reduction in communication overhead by a factor of $1,258\times$ to $80,536\times$ compared to SOTA interactive PNNS approaches, owing to its single-round communication design and efficient SIMD packing strategy. We further compare the query latency of HANNS with interactive PNNS schemes [23, 27] under varying network bandwidths. We observe that under limited bandwidth (e.g., 100 Kbps), HANNS achieves up to $8\times$ and $119\times$ lower latency than Panther [27] and SANNS [23], respectively. While interactive schemes perform better in high-bandwidth environments, their reliance on multi-round interactions requires the client to keep online throughout the protocol, increasing the burden on the client. Moreover, unlike HANNS, [27] and [23] do not support the outsourced computation scenarios, which further limits their applicability in practical cloud computing scenarios.

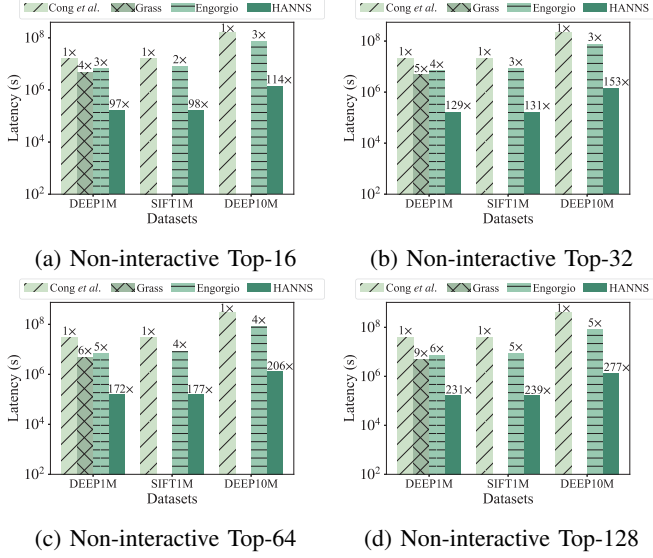


Figure 7: End-to-end performance comparisons between HANNS and non-interactive schemes across various datasets.

TABLE 5: Communication performance of HANNS and interactive PNNS schemes.

Dataset	Methods	Comm.	
SIFT1M	SANNS [23]	1.77GB	19,333×
	Panther [27]	118MB	1,258×
	This Work	96KB	1×
DEEP1M	SANNS [23]	1.58GB	23,010×
	Panther [27]	126MB	1,792×
	This Work	72KB	1×
DEEP10M	SANNS [23]	5.53GB	80,536×
	Panther [27]	318MB	4,522×
	This Work	72KB	1×

8. Conclusions

In this work, we present HANNS, an FHE-based PNNS framework that evaluates encrypted vector search queries in a non-interactive manner with a sublinear number of homomorphic comparisons. By introducing a novel cluster ordering scheme and a vector compression technique, HANNS enables non-interactive ANNS over large-scale, high-dimensional encrypted vectors. Our experimental results demonstrate that HANNS significantly outperforms both non-interactive and interactive PNNS solutions in terms of computational and communication efficiency. Moreover, HANNS supports both the outsourcing model and the two-party security model, offering flexibility for deployment in diverse practical scenarios.

References

[1] J. Suchal and P. Návrat, “Full text search engine as scalable k-nearest neighbor recommendation system,” in *IFIP Artificial Intelligence in Theory and Practice*, pp. 165–173, 2010.

[2] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[3] P. Lewis, E. Perez, A. Piktus, *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” *Advances in Neural Information Processing Systems*, vol. 33.

[4] A. Henzinger, E. Dauterman, H. Corrigan-Gibbs, and N. Zeldovich, “Private Web Search with Tiptoe,” in *SOSP*, p. 396–416, 2023.

[5] C. Wei, B. Wu, S. Wang, R. Lou, C. Zhan, F. Li, and Y. Cai, “Analyticdb-v: A hybrid analytical engine towards query fusion for structured and unstructured data,” *Proc. VLDB Endow.*, pp. 3152–3165, 2020.

[6] J. Wang, X. Yi, R. Guo, *et al.*, “Milvus: A purpose-built vector data management system,” in *SIGMOD*, pp. 2614–2627, 2021.

[7] Q. Zhang, S. Xu, Q. Chen, *et al.*, “VBASE: unifying online vector similarity search and relational queries via relaxed monotonicity,” in *USENIX Symposium on Operating Systems Design and Implementation*, pp. 377–395, 2023.

[8] T. Schneider and O. Tkachenko, “Towards efficient privacy-preserving similar sequence queries on out-sourced genomic databases,” in *Proceedings of the 2018 Workshop on Privacy in the Electronic Society*, p. 71–75, 2018.

[9] G. Asharov, S. Halevi, Y. Lindell, and T. Rabin, “Privacy-preserving search of similar patients in genomic data,” *Proc. Priv. Enhancing Technol.*, vol. 2018, no. 4, pp. 104–124, 2018.

[10] J. J. Pan, J. Wang, and G. Li, “Survey of vector database management systems,” *The VLDB Journal*, vol. 33, no. 5, pp. 1591–1615, 2024.

[11] AWS, “Machine Learning on AWS.” https://aws.amazon.com/machine-learning/?nc2=h_ql_sol_use_ml, 2023. Accessed: 2023-01-01.

[12] Azure, “Azure Machine Learning.” <https://azure.microsoft.com/en-us/products/machine-learning/>, 2023. Accessed: 2023-01-01.

[13] MongoDB, “Application-driven analytics.” <https://www.mongodb.com/use-cases/analytics>, 2023. Accessed: 2023-01-01.

[14] G. Cloud, “Cloud SQL.” <https://cloud.google.com/sql/>, 2023. Accessed: 2023-01-01.

[15] S. Tu, W. Zheng, E. Kohler, B. Liskov, and S. Madden, “Speedy transactions in multicore in-memory databases,” in *SOSP*, p. 18–32, 2013.

[16] W. Zheng, S. Tu, E. Kohler, and B. Liskov, “Fast Databases with Fast Durability and Recovery Through Multicore Parallelism,” in *OSDI*, 2014.

[17] L. J. Trautman and P. C. Ormerod, “Corporate directors’ and officers’ cybersecurity standard of care: The yahoo data breach,” *American University Law Review*, vol. 66, no. 5, p. 3, 2017.

[18] C. Cadwalladr and E. Graham-Harrison, “Revealed: 50 million facebook profiles harvested for cambridge

- analytica in major data breach,” *The guardian*, vol. 17, p. 22, 2018.
- [19] J. Zhu, L. Patel, M. Zaharia, and R. A. Popa, “Compass: Encrypted semantic search with high accuracy,” Cryptology ePrint Archive, Paper 2024/1255, 2024.
 - [20] M. Zuber and R. Sirdey, “Efficient homomorphic evaluation of k-nn classifiers,” *Proc. Priv. Enhancing Technol.*, vol. 2021, no. 2, pp. 111–129, 2021.
 - [21] S. Bian, H. Pan, J. Hu, Z. Zhang, Y. Fu, J. Hua, Y. Chen, B. Zhang, Y. Jin, J. Dong, and Z. Guan, “Engorgio: An arbitrary-precision unbounded-size hybrid encrypted database via quantized fully homomorphic encryption,” in *USENIX Security Symposium*, 2025.
 - [22] S. Hong, S. Kim, J. Choi, Y. Lee, and J. H. Cheon, “Efficient sorting of homomorphic encrypted data with k-way sorting network,” *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 4389–4404, 2021.
 - [23] H. Chen, I. Chillotti, Y. Dong, O. Poburinnaya, I. P. Razenshteyn, and M. S. Riazi, “SANNS: scaling up secure approximate k-nearest neighbors search,” in *USENIX Security Symposium*, pp. 2111–2128, 2020.
 - [24] S. Servan-Schreiber, S. Langowski, and S. Devadas, “Private approximate nearest neighbor search with sublinear communication,” in *IEEE SP*, pp. 911–929, 2022.
 - [25] K. Cong, R. Geelen, J. Kang, and J. Park, “Revisiting Oblivious Top- k Selection with Applications to Secure k -NN Classification,” Cryptology ePrint Archive, Paper 2023/852, 2023.
 - [26] D. Kim, Y. Nam, W. Wang, H. Gong, I. Bhati, R. Cammarota, T. S. Rosing, M. Tepper, and T. L. Willke, “GraSS: Graph-based similarity search on encrypted query,” Cryptology ePrint Archive, Paper 2024/1012, 2024.
 - [27] J. Li, Z. Huang, M. Zhang, J. Liu, C. Hong, T. Wei, and W. Chen, “PANTHER: Private approximate nearest neighbor search in the single server setting,” Cryptology ePrint Archive, Paper 2024/1774, 2024.
 - [28] M. Zhou, E. Shi, and G. Fanti, “Pacmann: Efficient private approximate nearest neighbor search,” Cryptology ePrint Archive, Paper 2024/1600, 2024.
 - [29] H. Shaul, D. Feldman, and D. Rus, “Secure k-ish nearest neighbors classifier,” *Proc. Priv. Enhancing Technol.*, pp. 42–61, 2020.
 - [30] A. Chatterjee, M. Kaushal, and I. Sengupta, “Accelerating sorting of fully homomorphic encrypted data,” in *INDOCRYPT 2013*, vol. 8250, pp. 262–273, 2013.
 - [31] B. Wang, Y. Hou, and M. Li, “Practical and secure nearest neighbor search on encrypted large-scale data,” in *IEEE International Conference on Computer Communications*, pp. 1–9, 2016.
 - [32] B. Yao, F. Li, and X. Xiao, “Secure nearest neighbor revisited,” in *IEEE International Conference on Data Engineering*, pp. 733–744, 2013.
 - [33] Y. Elmehdwi, B. K. Samanthula, and W. Jiang, “Secure k-nearest neighbor query over encrypted data in outsourced environments,” in *IEEE International Conference on Data Engineering*, pp. 664–675, 2014.
 - [34] N. Wang, W. Zhou, J. Wang, Y. Guo, J. Fu, and J. Liu, “Secure and efficient similarity retrieval in cloud computing based on homomorphic encryption,” *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 2454–2469, 2024.
 - [35] A. Boldyreva and T. Tang, “Privacy-preserving approximate k-nearest-neighbors search that hides access, query and volume patterns,” *Proc. Priv. Enhancing Technol.*, vol. 2021, pp. 549–574, 2021.
 - [36] A. C.-C. Yao, “How to generate and exchange secrets,” in *27th Annual Symposium on Foundations of Computer Science (sfcs 1986)*, 1986.
 - [37] N. Gilboa and Y. Ishai, “Distributed point functions and their applications,” in *Advances in Cryptology - EUROCRYPT 2014* (P. Q. Nguyen and E. Oswald, eds.), vol. 8441, pp. 640–658, 2014.
 - [38] H. Chen, I. Chillotti, and L. Ren, “Onion ring ORAM: efficient constant bandwidth oblivious RAM from (leveled) TFHE,” in *ACM CCS*, pp. 345–360, 2019.
 - [39] A. Henzinger, M. M. Hong, H. Corrigan-Gibbs, S. Meiklejohn, and V. Vaikuntanathan, “One server for the price of two: Simple and fast Single-Server private information retrieval,” in *USENIX Security Symposium*, 2023.
 - [40] E. Dauterman, M. Rathee, R. A. Popa, and I. Stoica, “Waldo: A Private Time-Series Database from Function Secret Sharing,” in *IEEE SP*, pp. 2450–2468, 2022.
 - [41] E. Boyle, N. Gilboa, and Y. Ishai, “Secure computation with preprocessing via function secret sharing,” in *Theory of Cryptography International Conference, TCC*, vol. 11891, pp. 341–371, 2019.
 - [42] E. Boyle, N. Chandran, N. Gilboa, *et al.*, “Function secret sharing for mixed-mode and fixed-point secure computation,” in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pp. 871–900, 2021.
 - [43] T. Araki, A. Barak, J. Furukawa, *et al.*, “Optimized honest-majority mpc for malicious adversaries—breaking the 1 billion-gate per second barrier,” in *IEEE SP*, pp. 843–862, 2017.
 - [44] F. Mazzone, M. Everts, F. Hahn, and A. Peter, “Efficient ranking, order statistics, and sorting under ckks,” in *34th USENIX Security Symposium*, (Seattle, WA), USENIX Association, aug 2025.
 - [45] N. Cui, X. Yang, B. Wang, J. Li, and G. Wang, “Svknn: Efficient secure and verifiable k-nearest neighbor query on the cloud platform*,” in *IEEE International Conference on Data Engineering*, pp. 253–264, 2020.
 - [46] Y. Zheng, R. Lu, Y. Guan, J. Shao, and H. Zhu, “Efficient and privacy-preserving similarity range query over encrypted time series data,” *IEEE TDSC*, vol. 19, no. 4, pp. 2501–2516, 2022.
 - [47] J. H. Cheon, K. Han, A. Kim, M. Kim, and Y. Song, “A Full RNS Variant of Approximate Homomorphic Encryption,” Cryptology ePrint Archive, Paper 2018/931, 2018.
 - [48] S. Halevi, Y. Polyakov, and V. Shoup, “An improved

- RNS variant of the BFV homomorphic encryption scheme,” in *CT-RSA*, pp. 83–105, 2019.
- [49] K. Han, M. Hhan, and J. H. Cheon, “Improved homomorphic discrete fourier transforms and FHE bootstrapping,” *IEEE Access*, vol. 7, pp. 57361–57370, 2019.
- [50] S. Halevi and V. Shoup, “Design and implementation of helib: a homomorphic encryption library,” *IACR Cryptol. ePrint Arch.*, 2020.
- [51] J. Bossuat, C. Mouchet, J. R. Troncoso-Pastoriza, and J. Hubaux, “Efficient Bootstrapping for Approximate Homomorphic Encryption with Non-sparse Keys,” in *EUROCRYPT*, vol. 12696, pp. 587–617, 2021.
- [52] J. H. Cheon, K. Han, A. Kim, M. Kim, and Y. Song, “Bootstrapping for Approximate Homomorphic Encryption,” in *EUROCRYPT*, vol. 10820, pp. 360–384, 2018.
- [53] Y. Lee, J. Lee, Y. Kim, Y. Kim, J. No, and H. Kang, “High-Precision Bootstrapping for Approximate Homomorphic Encryption by Error Variance Minimization,” in *EUROCRYPT*, vol. 13275, pp. 551–580.
- [54] O. Regev, “On lattices, learning with errors, random linear codes, and cryptography,” *J. ACM*, vol. 56, no. 6, p. 34, 2009.
- [55] V. Lyubashevsky, C. Peikert, and O. Regev, “On ideal lattices and learning with errors over rings,” in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pp. 1–23, 2010.
- [56] Y. Bae, J. H. Cheon, W. Cho, *et al.*, “META-BTS: Bootstrapping Precision Beyond the Limit,” in *ACM CCS*, pp. 223–234, 2022.
- [57] H. Chung, H. Kim, Y.-S. Kim, and Y. Lee, “Amortized large look-up table evaluation with multivariate polynomials for homomorphic encryption.” *Cryptology ePrint Archive*, Paper 2024/274, 2024.
- [58] A. Alexandru, A. Kim, and Y. Polyakov, “General functional bootstrapping using CKKS,” *IACR Cryptol. ePrint Arch.*, p. 1623, 2024.
- [59] Y. Bae, J. Kim, D. Stehlé, and E. Suvanto, “Bootstrapping small integers with ckks,” in *Advances in Cryptology – ASIACRYPT 2024*, p. 330–360, 2024.
- [60] Y. Bae, J. H. Cheon, J. Kim, and D. Stehlé, “Bootstrapping bits with ckks,” in *Advances in Cryptology – EUROCRYPT 2024*, p. 94–123, 2024.
- [61] N. Drucker, G. Moshkovich, T. Pelleg, and H. Shaul, “BLEACH: cleaning errors in discrete computations over CKKS,” *IACR Cryptol. ePrint Arch.*, p. 1298, 2022.
- [62] J. Kim and T. Noh, “Modular reduction in CKKS.” *Cryptology ePrint Archive*, Paper 2024/1638, 2024.
- [63] H. Chung, H. Kim, Y.-S. Kim, and Y. Lee, “Amortized large look-up table evaluation with multivariate polynomials for homomorphic encryption.” *Cryptology ePrint Archive*, Paper 2024/274, 2024.
- [64] J. Kim, “Efficient homomorphic integer computer from CKKS.” *Cryptology ePrint Archive*, Paper 2025/066, 2025.
- [65] D. Boneh and J. Kim, “Homomorphic encryption for large integers from nested residue number systems.” *Cryptology ePrint Archive*, Paper 2025/346, 2025.
- [66] Z. Brakerski, “Fully homomorphic encryption without modulus switching from classical gapsvp,” in *CRYPTO*, pp. 868–886, 2012.
- [67] J. H. Cheon, A. Kim, M. Kim, and Y. S. Song, “Homomorphic encryption for arithmetic of approximate numbers,” in *ASIACRYPT*, pp. 409–437, 2017.
- [68] A. Varma, “Trigonometric interpolation,” *Journal of Mathematical Analysis and Applications*, vol. 28, no. 3, pp. 652–659, 1969.
- [69] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.
- [70] H. Ootomo, A. Naruse, C. Nolet, R. Wang, T. Feher, and Y. Wang, “Cagra: Highly parallel graph construction and approximate nearest neighbor search for gpus,” in *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, pp. 4236–4247, 2024.
- [71] J. L. Bentley, “Multidimensional binary search trees used for associative searching,” *Commun. ACM*, vol. 18, no. 9, p. 509–517, 1975.
- [72] Y. Zheng, Q. Guo, A. K. Tung, and S. Wu, “Lazyish: Approximate nearest neighbor search for multiple distance functions with a single index,” in *Proceedings of the 2016 International Conference on Management of Data*, 2016.
- [73] Y. Sun, W. Wang, J. Qin, Y. Zhang, and X. Lin, “Srs: solving c-approximate nearest neighbor queries in high dimensional euclidean space with a tiny index,” *Proc. VLDB Endow.*, vol. 8, no. 1, 2014.
- [74] H. Jégou, M. Douze, and C. Schmid, “Product quantization for nearest neighbor search,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 1, pp. 117–128, 2011.
- [75] S. Lloyd, “Least squares quantization in pcm,” *IEEE Trans. Inf. Theor.*, vol. 28, no. 2, p. 129–137, 2006.
- [76] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, “The faiss library,” 2024.
- [77] Z. Zhang, S. Bian, Z. Zhao, R. Mao, H. Zhou, J. Hua, Y. Jin, and Z. Guan, “ArcEDB: An Arbitrary-Precision Encrypted Database via (Amortized) Modular Homomorphic Encryption,” in *ACM CCS*, p. 4613–4627, 2024.
- [78] R. A. Popa, C. M. S. Redfield, N. Zeldovich, and H. Balakrishnan, “CryptDB: protecting confidentiality with encrypted query processing,” in *ACM SOSR*, pp. 85–100, 2011.
- [79] T. Hackenjos, F. Hahn, and F. Kerschbaum, “SAGMA: Secure Aggregation Grouped by Multiple Attributes,” in *SIGMOD*, pp. 587–601, 2020.
- [80] Y. Kortekaas, “Access Pattern Hiding Aggregation over Encrypted Databases,” October 2020.
- [81] K. Cong, D. Das, J. Park, and H. V. L. Pereira,

- “SortingHat: Efficient Private Decision Tree Evaluation via Homomorphic Encryption and Transciphering,” in *ACM CCS*, pp. 563–577, 2022.
- [82] X. Ren, L. Su, Z. Gu, S. Wang, F. Li, Y. Xie, S. Bian, C. Li, and F. Zhang, “Heda: Multi-attribute unbounded aggregation over homomorphically encrypted database,” *Proc. VLDB Endow.*, vol. 16, no. 4, pp. 601–614, 2022.
- [83] S. Bian, Z. Zhang, H. Pan, R. Mao, Z. Zhao, Y. Jin, and Z. Guan, “HE3DB: an efficient and elastic encrypted database via arithmetic-and-logic fully homomorphic encryption,” in *ACM CCS*, pp. 2930–2944, 2023.
- [84] I. Chillotti, D. Ligier, J. Orfila, and S. Tap, “Improved programmable bootstrapping with larger precision and efficient arithmetic circuits for TFHE,” in *ASIACRYPT*, pp. 670–699, 2021.
- [85] J. Fan and F. Vercauteren, “Somewhat practical fully homomorphic encryption,” *IACR Cryptol. ePrint Arch.*, p. 144, 2012.
- [86] Z. Li, H. Li, and Z. Wang, “SIMD-style sorting of integer sequence in RLWE ciphertext,” *Cryptology ePrint Archive*, Paper 2024/1648, 2024.
- [87] S. Azogagh, M.-O. Killijian, and F. Larose-Gervais, “A non-comparison oblivious sort and its application to private k-NN,” *Cryptology ePrint Archive*, Paper 2024/1894, 2024.
- [88] K. E. Batchner, “Sorting networks and their applications,” in *American Federation of Information Processing Societies*, pp. 307–314, 1968.
- [89] R. Akhavan Mahdavi, H. Ni, D. Linkov, and F. Kerschbaum, “Level up: Private non-interactive decision tree evaluation using levelled homomorphic encryption,” in *ACM CCS*, p. 2945–2958, 2023.
- [90] K. Cong, J. Kang, G. Nicolas, and J. Park, “Faster private decision tree evaluation for batched input from homomorphic encryption,” *Cryptology ePrint Archive*, Paper 2024/662, 2024.
- [91] T. Rivlin, “The chebyshev polynomials, pure and applied mathematics,” 1974.
- [92] J. H. Cheon, D. Kim, D. Kim, H. Lee, and K. Lee, “Numerical method for comparison on homomorphically encrypted numbers,” in *ASIACRYPT*, pp. 415–445, 2019.
- [93] J. H. Cheon, D. Kim, and D. Kim, “Efficient homomorphic comparison methods with optimal complexity,” in *ASIACRYPT*, pp. 221–256, 2020.
- [94] E. Lee, J. Lee, J. No, and Y. Kim, “Minimax approximation of sign function by composite polynomial for homomorphic comparison,” *IEEE TDSC*, vol. 19, pp. 3711–3727, 2022.
- [95] J. H. Cheon, D. Kim, D. Kim, H. Lee, and K. Lee, “Numerical Method for Comparison on Homomorphically Encrypted Numbers,” in *ASIACRYPT*, vol. 11922, pp. 415–445, 2019.
- [96] A. A. Badawi, A. Alexandru, J. Bates, *et al.*, “OpenFHE: Open-source fully homomorphic encryption library,” *Cryptology ePrint Archive*, Paper 2022/915, 2022.

- [97] J. H. Cheon, Y. Son, and D. Yhee, “Practical FHE parameters against lattice attacks,” *IACR Cryptol. ePrint Arch.*, p. 39, 2021.
- [98] M. R. Albrecht, R. Player, and S. Scott, “On the concrete hardness of learning with errors,” *J. Math. Cryptol.*, pp. 169–203, 2015.
- [99] J.-P. Bossuat, R. Cammarota, J. H. Cheon, *et al.*, “Security Guidelines for Implementing Homomorphic Encryption,” *Cryptology ePrint Archive*, Paper 2024/463, 2024.
- [100] Zama, “TFHE-rs: A Pure Rust Implementation of the TFHE Scheme for Boolean and Integer Arithmetics Over Encrypted Data,” 2022. <https://github.com/zama-ai/tfhe-rs>.
- [101] I. Iliashenko and V. Zucca, “Faster homomorphic comparison operations for BGV and BFV,” *Proc. Priv. Enhancing Technol.*, vol. 2021, pp. 246–264, 2021.

Appendix A. Full Notations

We summarize the notations and operators used in this work in Table A1.

Appendix B. Bivariate Chebyshev Approximation

To approximate a bivariate function $f(x, y)$ using Chebyshev polynomials, we express it as tensor-product expansion:

$$f(x, y) \approx \sum_{i=0}^{N_x} \sum_{j=0}^{N_y} c_{i,j} T_i(x) T_j(y), \quad (\text{A1})$$

where $T_i(x)$ and $T_j(y)$ are i -th and j -th Chebyshev polynomials of the first kind, N_x and N_y are the approximation degree. Therefore, the main goal of bivariate Chebyshev approximation is to compute the coefficients $c_{i,j}$. The coefficients $c_{i,j}$ are computed via two-dimensional discrete inner product using Chebyshev–Gauss nodes:

$$x_k = \cos\left(\frac{\pi(k+0.5)}{N_x+1}\right), y_m = \cos\left(\frac{\pi(m+0.5)}{N_y+1}\right),$$

with uniform weights $w_{k,m} = \frac{1}{(N_x+1)(N_y+1)}$, where $0 \leq k \leq N_x, 0 \leq m \leq N_y$. The coefficient is then given by

$$\begin{aligned} c_{i,j} &= \frac{\langle f, T_i(x) T_j(y) \rangle}{\|T_i(x)\|^2 \cdot \|T_j(y)\|^2} \\ &= \frac{\sum_{k=0}^{N_x} \sum_{m=0}^{N_y} f(x_k, y_m) T_i(x_k) T_j(y_m) w_{k,m}}{\left(\frac{N_x+1}{2} \cdot 2^{\delta_{i,0}}\right) \left(\frac{N_y+1}{2} \cdot 2^{\delta_{j,0}}\right)}, \end{aligned}$$

which simplifies to

$$c_{i,j} = \frac{2^{\delta_{i,0}+\delta_{j,0}}}{(N_x+1)(N_y+1)} \sum_{k=0}^{N_x} \sum_{m=0}^{N_y} f(x_k, y_m) T_i(x_k) T_j(y_m),$$

where $\delta_{i,0}$ and $\delta_{j,0}$ are Kronecker delta functions used to correct the norm of T_0 .

Appendix Table A1: Summary of Notations.

Notation	Description
$\mathcal{D}$	The database
$\mathcal{PD}$	The database after preprocessing
$\mathbf{PV}$	The set of PQ-encoded vectors
$\mathcal{C}$	The set of cluster centroids
$\mathcal{CB}$	The PQ codebook
$[\mathcal{PD}]$	The encrypted preprocessed database
$[\mathbf{PV}]$	The encrypted PQ-encoded vectors
$[\mathcal{C}]$	The encrypted cluster centroids
$[\mathcal{CB}]$	The encrypted PQ codebook
$\mathcal{DT}$	The PQ distance table
$\mathcal{C}_i$	The i -th cluster
$\mathbf{c}_i$	The centroid of the i -th cluster
$\mathcal{C}$	The set of centroids
$\mathbf{v}_{i,j}$	The j -th vector in the i -th cluster
$\mathbf{r}_{i,j}$	The j -th residual vector in the i -th cluster
$\mathbf{b}_i^{(z)}$	The i -th PQ code in the z -th PQ subspace
$\mathbf{pv}_{i,j}$	The j -th PQ-encoded vector in the i -th cluster
id_z	The z -th PQ code in PQ-encoded vector $\mathbf{pv}_{i,j}$
$\mathbf{qv}$	The query vector
$\mathbf{q}^{(i)}$	The residual vector between $\mathbf{qv}$ and selected centroid $\mathbf{c}_i$
d	The degree of the vector
u	The number of clusters selected during the coarse search
k	The number of results finally returned
n	The size of the database
l	The size of each cluster
t	The number of clusters
l	The number of PQ subspaces
$\mathfrak{t}$	The codebook size of each subspace
λ	The security parameter
P	The plaintext modulus
Q	The ciphertext modulus for RLWE
N	The lattice dimension for an RLWE
$\mathbb{Z}_Q$	The set of n -vectors over $\mathbb{Z}_Q$
R	The cyclotomic ring $\mathbb{Z}[X]/(X^N + 1)$
R_Q	The cyclotomic ring $\mathbb{Z}_Q[X]/(X^N + 1)$
χ	The noise distribution
Δ	The scaling factor
$\mathbf{y}$	An element in vector domain
$\mathbf{y}^{[i]}$	The i -th element of $\mathbf{y}$
$\tilde{y}$	An element in polynomial ring
$\tilde{y}^{[i]}$	The i -th coefficient of $\tilde{y}$
$\mathbf{A}$	An element in matrix domain
ζ	The quantization precision
w	The number of quantized segments
B_g	The quantized radix base
$\hat{\mathbf{m}}$	The quantized polynomial
$\hat{\hat{\mathbf{m}}}$	A collection of segmented polynomials

Due to the inherent discontinuity of the sign function, approximate comparisons under CKKS typically require the composition of multiple functions to achieve acceptable accuracy [21, 92–94, 101]. For such composite functions, we observe that the evaluation order can be flexibly adjusted to improve both efficiency and approximation quality. Specifically, we first approximate the higher-order composite function using a bivariate Chebyshev expansion. Once the result of this high-order component is obtained, it is used as input to compute additional lower-order terms. To evaluate composite functions of the form $g(f(x - y))$, where $f(x - y)$ is approximated using bivariate Chebyshev

polynomials as Equation (A1). To evaluate the composite function $g(f(x - y)) = f(x - y)^k$, we begin with the square case:

$$[f(x - y)]^2 = \left(\sum_{i,j} \mathbf{C}_{i,j} T_i(x) T_j(y) \right) \left(\sum_{k,l} \mathbf{C}_{k,l} T_k(x) T_l(y) \right),$$

which expands into:

$$[f(x - y)]^2 = \sum_{i,j,k,l} \mathbf{C}_{i,j} \mathbf{C}_{k,l} T_i(x) T_k(x) T_j(y) T_l(y).$$

Using the Chebyshev product identity:

$$T_m(z) T_n(z) = \frac{1}{2} (T_{m+n}(z) + T_{|m-n|}(z)), \quad z \in \{x, y\},$$

each product term can be expressed as:

$$T_i(x) T_k(x) = \sum_{p=0}^{i+k} \mathbf{A}_{i,k,p} T_p(x),$$

$$T_j(y) T_l(y) = \sum_{q=0}^{j+l} \mathbf{B}_{j,l,q} T_q(y),$$

where $\mathbf{A}_{i,k,p}$ and $\mathbf{B}_{j,l,q}$ denote coefficients determined by the product rule.

Substituting into the previous expansion, we obtain:

$$[f(x - y)]^2 = \sum_{p=0}^{2N} \sum_{q=0}^{2M} \mathbf{D}_{p,q} T_p(x) T_q(y),$$

where the new coefficients $\mathbf{D}_{p,q}$ are computed as:

$$\mathbf{D}_{p,q} = \sum_{i=0}^N \sum_{k=0}^N \sum_{j=0}^M \sum_{l=0}^M \mathbf{C}_{i,j} \mathbf{C}_{k,l} \mathbf{A}_{i,k,p} \mathbf{B}_{j,l,q}.$$

This approach can be extended to higher powers $f(x - y)^k$ through recursive applications of the Chebyshev multiplication rule, enabling efficient approximation of composite functions $g(f(x - y))$ within the Chebyshev basis.

Appendix C. Homomorphic Order Application

To homomorphically permute ciphertext vectors according to the bitonic sorting network, we implement a lightweight matrix-based permutation operator HomOrdApp . This operator applies a sparse permutation matrix derived from comparison results in the bitonic sorting network to an encrypted input vector.

For each stage s and part p of the bitonic sorting network, we construct a corresponding permutation matrix $\mathbf{M}_{s,p}$. Each matrix is symmetric and sparse, with nonzero entries only on the main diagonal and its two adjacent diagonals. These values are extracted and encoded into three plaintext vectors:

$$\mathbf{v}_{\text{diag}} = \text{diag}(\mathbf{M}_{s,p}),$$

$$\mathbf{v}_{\text{sub}} = [\text{diag}_{-1}(\mathbf{M}_{s,p}), 0],$$

$$\mathbf{v}_{\text{sup}} = [0, \text{diag}_{+1}(\mathbf{M}_{s,p})].$$

Algorithm 6: Order Application HomOrdApp

Input : A QRLWE ciphertext $ct_{\tilde{x}}$, an encoded permutation matrix $\mathbf{CM}$, stage s , part p , and rotate key $\mathbf{RK}$.

Output : A QRLWE ciphertext ct_{res} , which encrypts the sort result of stage s , part p .

```
1 for  $i = 0$  to  $w - 1$  do
2    $ct_{\text{sup}}[i] \leftarrow \text{Rot}(ct_{\tilde{x}}[i], \mathbf{RK}, -2^{s-p})$ 
3    $ct_{\text{sub}}[i] \leftarrow \text{Rot}(ct_{\tilde{x}}[i], \mathbf{RK}, 2^{s-p})$ 
4    $ct_{\text{res}}[i] \leftarrow ct_{\text{sup}}[i] \cdot \mathbf{CM}_{s,p}[0] + ct_{\tilde{x}}[i] \cdot$   

    $\mathbf{CM}_{s,p}[1] + ct_{\text{sub}}[i] \cdot \mathbf{CM}_{s,p}[2]$ 
5 end
Return :  $ct_{\text{res}}$ 
```

The homomorphic permutation is computed by combining the original ciphertext vector with its left and right rotations:

$$\mathbf{v} \cdot \mathbf{M}_{s,p} = \mathbf{v} \circ \mathbf{v}_{\text{diag}} + (\mathbf{v} \ll 1) \circ \mathbf{v}_{\text{sub}} + (\mathbf{v} \ll (N - 1)) \circ \mathbf{v}_{\text{sup}},$$

where $\circ$ denotes pointwise multiplication and $\ll$ denotes homomorphic rotation.

Algorithm 6 shows the implementation of HomOrdApp , which performs two ciphertext rotations and three homomorphic multiplications per ciphertext.